Reduced ML with extern types and cast-capable operations

This formal summary captures the syntax, static semantics, and dynamic behavior of **Reduced ML** extended with the **extern** boundary and the cast-capable `let` binding.

1 Syntax

1.1 Types (τ) and Values (v)

The system includes standard ML types, products, a "boxed" foreign type called extern, and a runtime wrapper for foreign data.

- **Types:** $\tau ::= \text{unit} \mid \text{bool} \mid \text{int} \mid \tau \to \tau \mid \tau_1 \times \tau_2 \mid \tau\,\text{ref} \mid \text{extern}$
- **Expressions:** $e ::= () \mid \mathsf{true} \mid \mathsf{false} \mid i \mid x \mid \lambda x{:}\tau.\, e \mid e\ e \mid (e_1, e_2) \mid \text{fst}\ e \mid \text{snd}\ e \mid \text{if}\ e\ \text{then}\ e\ \text{else}\ e \mid \text{ref}\ e \mid !e \mid e := e \mid \text{cast}_\tau\, e \mid \phi \mid \text{let}\ x{:}\tau = e\ \text{in}\ e$
- **Values:** $v ::= () \mid \mathsf{true} \mid \mathsf{false} \mid i \mid l \mid \lambda x{:}\tau.\, e \mid (v_1, v_2) \mid \phi \mid \text{box}(v, \tau)$

We say that a type is *native* if it has no occurrences of the extern type in its composition.

Boundary discipline (cast-centric). There are no surface-only markers such as $\texttt{let}^{\mathsf{ext}}$ or application/assignment superscripts. Extern-compatible typing rules ([Let-extern], [App-extern], [Assign-extern]) use $\sim_{\mathsf{ext}}$ with the *right-hand* type always native; operationally, the expected native type is enforced via cast_τ and the conversion judgment $v \Downarrow_\tau v'$ (Section 3).

2 Static Typing Rules ($\Gamma \vdash e : \tau$)

> The key idea is that the extern type is only used at the interface between the FFI and the program. As soon as the foreign function (returning an extern) is called, the extern type is no longer relevant because the result will be cast to the expected type.

We generalize to *extern-compatible data* types (no function arrows): Let

$$\delta ::= \text{unit} \mid \text{bool} \mid \text{int} \mid \delta_1 \times \delta_2 \mid \delta\,\text{ref}$$

be native data types. We write extern-compatibility as $\tau' \sim_{\mathsf{ext}} \tau$ with the *right-hand* τ always a native data type δ (no extern in τ). Function types $\tau_1 \to \tau_2$ are excluded from this data-only relation.

Judgments use a *typing context* Γ mapping term variables in scope to types; for example, rule [Var] requires $x : \tau \in \Gamma$, and rules for λ and `let` extend Γ when typing bodies. Foreign constants ϕ are taken from a fixed global assignment of types (rule [Foreign-Const]: for each ϕ, a matching τ is fixed once for the whole program).

2.1 Core Rules (uniform for native and extern-compatible types)

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\,[\text{Var}] \qquad \frac{i \in \mathbb{Z}}{\Gamma \vdash i : \text{int}}\,[\text{Int}] \qquad \frac{}{\Gamma \vdash () : \text{unit}}\,[\text{Unit}] \qquad \frac{}{\Gamma \vdash \mathsf{true} : \text{bool}}\,[\text{True}] \qquad \frac{}{\Gamma \vdash \mathsf{false} : \text{bool}}\,[\text{False}]$$

$$\frac{\Gamma, x{:}\tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x{:}\tau_1.\, e : \tau_1 \to \tau_2}\,[\text{Abs}] \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\,[\text{App}]$$

Rule [App-extern] below generalizes [App] when the argument is extern-compatible with the domain.

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2}\,[\text{Pair}]$$

$$\frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst}\ e : \tau_1}\,[\text{Fst}] \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd}\ e : \tau_2}\,[\text{Snd}]$$

$$\frac{\Gamma \vdash e_0 : \text{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if}\ e_0\ \text{then}\ e_1\ \text{else}\ e_2 : \tau}\,[\text{If}]$$

$$\frac{\Gamma \vdash e : \tau\,\text{ref}}{\Gamma \vdash !e : \tau}\,[\text{Deref}]$$

$$\frac{\Gamma \vdash e_1 : \tau\,\text{ref} \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 := e_2 : \text{unit}}\,[\text{Assign}]$$

References only store native content types ([Ref]). Dereference and native assignment ([Assign]) use that type uniformly. Extern-compatible values may still be *written* when they match the cell's native type via $\sim_{\mathsf{ext}}$ ([Assign-extern]).

2.2 State and Foreign Typing Rules (extern-aware)

$$\frac{\Phi(\phi) = \tau \to \text{extern} \quad \tau \text{ is a native type}}{\Gamma \vdash \phi : \tau \to \text{extern}}\,[\text{Foreign-Const}]$$

where Φ fixes one arrow type per foreign constant. Extern-typed values flow only from these constants (and compositions thereof) at the FFI boundary; consuming native code meets them through cast_τ or extern-compatible rules.

$$\frac{\Gamma \vdash e : \tau \quad \tau \text{ is a native type}}{\Gamma \vdash \text{ref}\ e : \tau\,\text{ref}}\,[\text{Ref}]$$

$$\frac{\Gamma \vdash e_1 : \tau\,\text{ref} \quad \Gamma \vdash e_2 : \tau' \quad \tau' \sim_{\mathsf{ext}} \tau \quad \tau \text{ is a native type}}{\Gamma \vdash e_1 := e_2 : \text{unit}}\,[\text{Assign-extern}]$$

Creating a reference of a non-native type is disallowed via the type system. The assignment operation is only applicable to references of native types, but the assigned values can be non-native and extern-compatible. The assignment operation is semantically a generalization of the the assignment operation for native types because it will insert a runtime type check to ensure that the assigned value is compatible with the expected type. In the case of native types, the cast is trivial and can be omitted (compiler optimization).

$$\frac{\Gamma \vdash e_1 : \tau_1' \quad \tau_1' \sim_{\mathsf{ext}} \tau_1 \quad \Gamma, x{:}\tau_1 \vdash e_2 : \tau_2 \quad \tau_1 \text{ is a native type}}{\Gamma \vdash \text{let}\ x{:}\tau_1 = e_1\ \text{in}\ e_2 : \tau_2}\,[\text{Let-extern}]$$

The rule for let ensures that no non-native typed values exist in the body of the let binding. This stops the 'extern' escape hatch from being used to smuggle non-native typed values into the body of the let binding. If the bound value is native then the cast is trivial and can be omitted. This on-the-fly conversion of non-native to native values can also happen, most generally, in function applications:

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1' \quad \tau_1' \sim_{\mathsf{ext}} \tau_1 \quad \tau_1 \text{ is a native type}}{\Gamma \vdash e_1\ e_2 : \tau_2} \text{[App-extern]}$$

Rule [App-extern] generalizes [App] when the argument type is extern-compatible with the native domain τ_1; reflexivity of $\sim_{\mathsf{ext}}$ on native τ recovers ordinary application.

$$\frac{\Gamma \vdash e : \tau' \quad \tau' \sim_{\mathsf{ext}} \tau \quad \tau \text{ is a native type}}{\Gamma \vdash \mathrm{cast}_\tau\, e : \tau} \text{[Cast-Extern-Compatible]}$$

The extern-compatible typing rules insert cast_τ at elaboration toward the native type τ named in the rule: **[Let-extern]** as let $x{:}\tau = \mathrm{cast}_\tau\, e_1$ in e_2, **[App-extern]** as $e_1\ (\mathrm{cast}_{\tau_1}\, e_2)$, and **[Assign-extern]** as $e_1 := \mathrm{cast}_\tau\, e_2$, where in each case the outer τ (resp. τ_1) is the *native* type on the right of $\sim_{\mathsf{ext}}$.

3 Operational Semantics

Reduction relates configurations to configurations or to a distinguished outcome error:

$$\langle e, \mu\rangle \to \langle e', \mu'\rangle \quad \text{or} \quad \langle e, \mu\rangle \to \text{error}.$$

We take e to be the expression and μ the store (mapping locations l to values v). Boundary conversions are implemented by $\mathrm{cast}_\tau\, e$ and the partial conversion relation $v \Downarrow_\tau v'$ below, where τ is always native in cast and let rules.

3.1 Administrative Reductions

$$\begin{aligned}
\langle (\lambda x{:}\tau.\, e)\ v, \mu\rangle &\to \langle e[v'/x], \mu\rangle \quad (\text{if } v \Downarrow_\tau v') \\
\langle \mathrm{ref}\ v, \mu\rangle &\to \langle l, \mu[l \mapsto v]\rangle \quad (l \notin \mathrm{dom}(\mu)) \\
\langle !l, \mu\rangle &\to \langle \mu(l), \mu\rangle \\
\langle l := v, \mu\rangle &\to \langle (), \mu[l \mapsto v']\rangle \quad (\text{if } v \Downarrow_\tau v') \\
\langle \text{if } \mathsf{true} \text{ then } e_1 \text{ else } e_2, \mu\rangle &\to \langle e_1, \mu\rangle \\
\langle \text{if } \mathsf{false} \text{ then } e_1 \text{ else } e_2, \mu\rangle &\to \langle e_2, \mu\rangle
\end{aligned}$$

The ref rule allocates a fresh location; no auxiliary runtime type map is maintained.

3.2 Products and Projections

$$\begin{aligned}
\langle \mathrm{fst}\ (v_1, v_2), \mu\rangle &\to \langle v_1, \mu\rangle \\
\langle \mathrm{snd}\ (v_1, v_2), \mu\rangle &\to \langle v_2, \mu\rangle
\end{aligned}$$

Evaluation commutes into pairs and projections when the relevant sub-expression steps (left-to-right order for pairs):

$$\frac{\langle e_1, \mu\rangle \to \langle e_1', \mu'\rangle}{\langle (e_1, e_2), \mu\rangle \to \langle (e_1', e_2), \mu'\rangle} \text{[R-Pair-L]} \qquad \frac{\langle e_2, \mu\rangle \to \langle e_2', \mu'\rangle}{\langle (v_1, e_2), \mu\rangle \to \langle (v_1, e_2'), \mu'\rangle} \text{[R-Pair-R]}$$

$$\frac{\langle e, \mu\rangle \to \langle e', \mu'\rangle}{\langle \mathrm{fst}\ e, \mu\rangle \to \langle \mathrm{fst}\ e', \mu'\rangle} \text{[R-Fst]} \qquad \frac{\langle e, \mu\rangle \to \langle e', \mu'\rangle}{\langle \mathrm{snd}\ e, \mu\rangle \to \langle \mathrm{snd}\ e', \mu'\rangle} \text{[R-Snd]}$$

3.3 Application, references, and assignment (call-by-value)

$$\frac{\langle e_1, \mu\rangle \to \langle e_1', \mu'\rangle}{\langle e_1\ e_2, \mu\rangle \to \langle e_1'\ e_2, \mu'\rangle}\ [\text{R-App-L}] \qquad \frac{\langle e_2, \mu\rangle \to \langle e_2', \mu'\rangle}{\langle v_1\ e_2, \mu\rangle \to \langle v_1\ e_2', \mu'\rangle}\ [\text{R-App-R}]$$

$$\frac{\langle e, \mu\rangle \to \langle e', \mu'\rangle}{\langle \text{ref}\ e, \mu\rangle \to \langle \text{ref}\ e', \mu'\rangle}\ [\text{R-Ref}] \qquad \frac{\langle e, \mu\rangle \to \langle e', \mu'\rangle}{\langle !e, \mu\rangle \to \langle !e', \mu'\rangle}\ [\text{R-Deref}]$$

$$\frac{\langle e_1, \mu\rangle \to \langle e_1', \mu'\rangle}{\langle e_1 := e_2, \mu\rangle \to \langle e_1' := e_2, \mu'\rangle}\ [\text{R-Assign-L}] \qquad \frac{\langle e_2, \mu\rangle \to \langle e_2', \mu'\rangle}{\langle v_1 := e_2, \mu\rangle \to \langle v_1 := e_2', \mu'\rangle}\ [\text{R-Assign-R}]$$

3.4 Conditionals

The guard is evaluated first; branches are not reduced until the guard is a boolean value.

$$\frac{\langle e_0, \mu\rangle \to \langle e_0', \mu'\rangle}{\langle \text{if}\ e_0\ \text{then}\ e_1\ \text{else}\ e_2, \mu\rangle \to \langle \text{if}\ e_0'\ \text{then}\ e_1\ \text{else}\ e_2, \mu'\rangle}\ [\text{R-If}]$$

3.5 Foreign and Cast Transitions

Each foreign constant ϕ is a value and does not reduce. Booleans are values as usual. Define a partial conversion relation on values, $v \Downarrow_\tau v'$, only when τ is a native type. The phrase "conversion succeeds" means that $v \Downarrow_\tau v'$ is derivable from the following clauses (no other base cases):

$$\frac{}{() \Downarrow_{\text{unit}} ()} \qquad \frac{i \in \mathbb{Z}}{i \Downarrow_{\text{int}} i} \qquad \frac{}{\mathsf{true} \Downarrow_{\text{bool}} \mathsf{true}} \qquad \frac{}{\mathsf{false} \Downarrow_{\text{bool}} \mathsf{false}}$$

$$\frac{}{\lambda x{:}\tau_1.\, e \Downarrow_{\tau_1 \to \tau_2} \lambda x{:}\tau_1.\, e} \qquad \frac{}{l \Downarrow_{\tau\ \text{ref}} l} \qquad \frac{\Phi(\phi) = \tau \to \text{extern}}{\phi \Downarrow_{\tau \to \text{extern}} \phi}$$

$$\frac{v_1 \Downarrow_{\tau_1} v_1' \quad v_2 \Downarrow_{\tau_2} v_2'}{(v_1, v_2) \Downarrow_{\tau_1 \times \tau_2} (v_1', v_2')}$$

$$\frac{v \Downarrow_\tau v' \quad \sigma = \tau}{\text{box}(v, \sigma) \Downarrow_\tau v'}$$

If no such derivation exists, write $v \not\Downarrow_\tau$. Tags on $\text{box}(v, \sigma)$ must match the target τ; otherwise conversion fails.

Cast reduction rules:

$$\frac{v \Downarrow_\tau v'}{\langle \text{cast}_\tau\ v, \mu\rangle \to \langle v', \mu\rangle}\ [\text{R-Cast-Success}] \qquad \frac{v \not\Downarrow_\tau}{\langle \text{cast}_\tau\ v, \mu\rangle \to \text{error}}\ [\text{R-Cast-Fail}]$$

$$\frac{\langle e, \mu\rangle \to \langle e', \mu'\rangle}{\langle \text{cast}_\tau\ e, \mu\rangle \to \langle \text{cast}_\tau\ e', \mu'\rangle}\ [\text{R-Cast}]$$

Let reduction rules:

$$\frac{v \Downarrow_\tau v'}{\langle \text{let}\ x{:}\tau = v\ \text{in}\ e_2, \mu\rangle \to \langle e_2[v'/x], \mu\rangle}\ [\text{R-Let-Val}] \qquad \frac{v \not\Downarrow_\tau}{\langle \text{let}\ x{:}\tau = v\ \text{in}\ e_2, \mu\rangle \to \text{error}}\ [\text{R-Let-Val-Fail}]$$

$$\frac{\langle e_1, \mu\rangle \to \langle e_1', \mu'\rangle}{\langle \text{let}\ x{:}\tau = e_1\ \text{in}\ e_2, \mu\rangle \to \langle \text{let}\ x{:}\tau = e_1'\ \text{in}\ e_2, \mu'\rangle}\ [\text{R-Let}]$$

3.6 Blame (optional)

The rules above reduce mismatches at the FFI boundary to error without saying *which* party violated the agreement. Following work on *contracts* and *gradual typing*, *blame* names that responsibility: each runtime check is a contract between a *native* expectation fixed by the *static* rules (and elaboration) and a *foreign* or producer-side claim carried by the tag on $\mathrm{box}(v, \sigma)$. Concretely: for [Let-extern], the expectation is the native annotation τ on x (rules [R-Let-Val] / [R-Let-Val-Fail] and cast_τ after elaboration). For [Assign-extern], the expectation is the native content type τ such that the left-hand reference is typed at $\tau\,\mathrm{ref}$; elaboration wraps the right-hand side in cast_τ, and the administrative assignment step applies $v \Downarrow_\tau v'$ to the value v actually written (Section 3). Configurations $\langle e, \mu\rangle$ do not attach that τ to locations in μ—the calculus leaves τ as the native type supplied by the relevant typing derivation—but for typable programs those usages of τ agree across [Assign], [Assign-extern], elaboration, and reduction.

Two boundary checks can produce error today:

- `let` cast failure: the boxed value fails cast_τ (shape incompatibility or failed conversion). A typical policy is to *blame the producer* of the boxed value if the foreign side is obliged to match the callee's declared τ, and to *blame the native annotation* if τ was chosen incorrectly against a trustworthy foreign tag. Implementations fix one convention for diagnostics.
- **Assignment** cast failure: either cast_τ fails on the elaborated RHS ([R-Cast-Fail]), or the cast succeeds but the administrative assignment rule fails because $v \not\Downarrow_\tau$ for the value v reaching $l := v$. Either way the native τ is the one from [Assign] or [Assign-extern] for that reference. Mismatch usually *blames the writer* of the incompatible external value (or incorrect typing/elaboration that picked τ for the cell vs. the written shape).

This calculus does not decorate error with blame labels in the reduction relation; doing so would replace error by something like error_β for a label β drawn from a fixed set (e.g. `foreign`, `native`, or module paths). Foreign calls and higher-order boundaries could add additional labeled checks (by analogy with higher-order contracts); this fragment leaves them implicit. Conceptually, blame refines *who* failed, not *when*: the failures above still arise at the first reduction step where the offending cast_τ, `let`, or assignment boundary runs, not after unbounded deferred checking.

Example 1 (blame at `let`). Suppose native code expects an integer but receives a boxed boolean tag from the boundary:

$$\langle \mathrm{let}\ x{:}\mathrm{int} = \mathrm{box}(\mathsf{true}, \mathrm{bool})\ \mathrm{in}\ x, \mu\rangle \to \mathrm{error}.$$

Here $\sigma = \mathrm{bool}$ and $\tau = \mathrm{int}$ disagree. Under a "foreign-must-match-native" policy, blame is assigned to the producer of $\mathrm{box}(\mathsf{true}, \mathrm{bool})$; under a "trust-the-tag" policy, blame is assigned to the native annotation $x{:}\mathrm{int}$.

Example 2 (blame at assignment). After elaboration of [Assign-extern], writing an ill-typed boxed value into an `int` cell becomes $\mathrm{cast}_{\mathrm{int}}$ on the RHS, then an administrative write that requires $\Downarrow_{\mathrm{int}}$ on the resulting value. Suppose $\Gamma \vdash l : \mathrm{int\,ref}$ so $\tau = \mathrm{int}$ throughout:

$$\langle l := \mathrm{cast}_{\mathrm{int}}\ \mathrm{box}(\mathsf{false}, \mathrm{bool}), \mu\rangle \to^* \mathrm{error}.$$

Here the failure is typically at $\mathrm{cast}_{\mathrm{int}}$ ([R-Cast-Fail]): the tag σ on the box does not match the target native type τ. Thus $\sigma = \mathrm{bool}$ and $\tau = \mathrm{int}$ disagree. The usual reading blames the writer of the incompatible boxed value (or a typing/elaboration mistake that treated this cell as $\mathrm{int\,ref}$).

Note: The blame example is optional and only used for interoperability reasoning. I am not personally convinced that the concept of blame is useful for the purpose of this proposal.

4 Conclusions and Observations

The claims below follow from the typing rules (Section 2) and the operational semantics (Section 3) *as written*: configurations are $\langle e, \mu \rangle$ without a runtime type component in the store. Runtime checks use the native type τ named in those rules—annotations on cast-capable `let`, the domain in [App-extern] after elaboration, the content type τ in [Assign] / [Assign-extern], and conversion $v \Downarrow_\tau v'$ at λ-application and assignment. For well-typed programs, the τ at an assignment is the native content type of the LHS τ ref; [Assign-extern] inserts cast_τ so the same τ governs both the cast chain and the final store conversion, even though μ records only values, not types.

4.1 Can extern camouflage an ill-typed program?

One might fear that a program rejected by the typechecker could be made acceptable by *replacing* some annotations or argument types with extern, in the spirit of a universal escape hatch. That is not how this calculus behaves.

Rule [Let-extern] requires the annotation τ_1 to be a *native* type, so one cannot annotate the bound variable with extern: a `let` of the form "`let` x : extern $= e_1$ `in` e_2" is not derivable, and one cannot "fix" a `let` by widening the bound name's type to extern. The premise $\tau_1' \sim_{\mathsf{ext}} \tau_1$ allows the right-hand side to carry extern inside τ_1' while x keeps the native shape τ_1—controlled interoperability, not replacement of τ_1 by extern.

For λ-abstractions, rule [Abs] allows a parameter type τ_1 that mentions extern, so syntactically one can rewrite $\lambda x{:}\tau.\, e$ into $\lambda x{:}\text{extern}.\, e$. That can turn a rejected term into a derivable one only in *degenerate* situations: the old parameter type was not semantically constraining the body (unused parameter, identity, or similar). There is no subtyping rule making extern a subtype of `int` (or analogous): if $f : \text{int} \to \tau$, then $f\ x$ requires $x : \text{int}$ by rule [App], so $\lambda x{:}\text{bool}.\, f\ x$ is ill-typed; rewriting to $\lambda x{:}\text{extern}.\, f\ x$ still leaves $f\ x$ ill-typed in the body. So extern is not a general mechanism for silencing type errors the way a fully dynamic discipline allows.

This contrasts with mechanisms such as C#'s `dynamic` type: the compiler deliberately defers member resolution, overload choice, and many arity checks to run time, so a great deal of code type-checks at compile time even though analogous code over a concrete static type would be rejected, and failures may surface only when the program runs. The same idea is easy to sketch in *pseudo-ML* (not any concrete dialect—hypothetical `dynamic` and `invoke` stand in for deferred checking):

```
let d : dynamic = fetch_some_object () in
  invoke d "NoSuchMethod"   (* elides static checks; may fail at run time *)
end
```

Nothing analogous is available for arbitrary core expressions here: extern marks foreign *data at the FFI boundary*, not "defer all checking for this subexpression."

4.2 Degenerate rewrites of lambdas alone

Rule [Abs] does not require the parameter type τ_1 to be native, whereas [Let-extern] and [Ref] do constrain annotations and cell content types. At first glance this might suggest a way to rewrite a program so that extern appears where a native type used to be, effectively sidestepping the discipline enforced elsewhere—beyond the repair question above. That reading would still be misleading for any program that does real work on its arguments.

The typable examples of such a replacement are essentially the same degenerate cases: the native parameter type was not doing any real work in the body. If the parameter is unused,

its declared type is irrelevant to typechecking the body; if the body is only the identity, the parameter type is carried through uniformly and can be renamed coherently. For instance,

$$\lambda x{:}\mathrm{int}.\,() \;\rightsquigarrow\; \lambda x{:}\mathrm{extern}.\,(), \qquad \lambda x{:}\mathrm{int}.\,x \;\rightsquigarrow\; \lambda x{:}\mathrm{extern}.\,x.$$

As soon as the body uses the parameter in a way that commits to a particular native structure—arithmetic, projections, dereferencing, application with a fixed domain, or a boolean guard of `if`—replacing a native type by extern breaks the derivation, just as in ordinary statically typed ML. The FFI-specific rules (`let`, `box`, assignment) remain the controlled entry points for extern when native code *consumes* foreign data; [Abs] alone does not open a back door for smuggling foreign values into code that expects real native structure in the body.

We do *not* add to [Abs] the requirement that τ_1 be native. Interoperability runs in *both* directions: native code calls foreign routines (values flow out and back through extern), but foreign code also calls into native routines. When the callee is written in the core language and invoked from outside, arguments are how foreign data enters the native function; those formals naturally carry types that mention extern at the boundary until the body casts or unpacks them (e.g. via `let`). Forcing every λ-bound parameter to be native would rule out such entry points and would mistake a presentation preference for a semantic requirement. Rules [Let-extern] and [Ref] keep storage and let-bound names on native types where the design intends a fixed ML shape; [Abs] stays more liberal precisely to model callbacks and other "native code called with foreign arguments" scenarios.

4.3 No flow-sensitive typing on branches

Rule [If] checks both branches under the *same* context Γ and requires them to share the same result type τ. There is no side condition that updates Γ after the guard—for example, no rule that refines a variable's type in the `then` branch and assigns a different refinement in the `else` branch. Consequently, *flow-sensitive* typing in the sense of "the same program variable has one static type along one branch and another static type along the other" is impossible here: whatever $x : \tau'$ appears in Γ applies uniformly to both arms.

Introducing extern does not change that picture. It widens what may flow *into* certain forms (e.g. `let` and assignment) from the FFI, but it does not add type refinements or branch-indexed contexts, so one cannot statically treat x as `int` on one path and `bool` on another while keeping a single binding in Γ.

This contrasts sharply with *dynamic typing* (e.g. Lisp, Python, untyped JavaScript), where a name typically denotes a single runtime slot whose *value* can hold different shapes at different times, and branch tests are often used to decide how to use that value at run time. Errors are deferred until an operation fails on the actual tag. Here, by contrast, each variable has one compile-time type for the purpose of checking a whole subexpression; `if` does not fork the static description of the environment. The extern boundary adds controlled checks at explicit FFI points—cast_τ on `let`, application arguments, and assignment (after elaboration), plus the conversion judgment where rules demand it—without a flow-sensitive or dynamically typed core.

5 Type inference in the presence of extern

The typing rules above are declarative: they do not specify an algorithm. This section records how extern interacts with the usual goal of *type inference*—recovering annotations automatically while respecting the same discipline—as one would implement for a language based on this calculus.

5.1 Where inference is routine

In subexpressions whose types are fixed by the rules up to equational constraints, extern behaves like any other nullary type constructor (e.g. `int` or `bool`) for the purposes of *unification*. For example, if $\Gamma \vdash e_1 : \tau_1 \to \tau_2$ and $\Gamma \vdash e_2 : \tau_1$ under unknowns for type variables, rule [App] is unchanged when τ_2 or τ_1 mentions extern. Foreign constants ϕ with $\Gamma \vdash \phi : \tau \to \text{extern}$ in the fixed global assignment contribute known function types, just like primitives.

5.2 The cast-capable `let` and missing annotations

Rule [Let-extern] is the delicate case for inference. It concludes $\Gamma \vdash \texttt{let}\ x{:}\tau_1 = e_1\ \texttt{in}\ e_2 : \tau_2$ when τ_1 is native, $\Gamma, x{:}\tau_1 \vdash e_2 : \tau_2$, and $\Gamma \vdash e_1 : \tau_1'$ with $\tau_1' \sim_{\mathsf{ext}} \tau_1$.

If the implementer drops the annotation τ_1 and tries to infer it *only* from e_1, the situation is underdetermined: from e_1 alone one may learn only some extern-compatible τ_1', which does not fix which native τ_1 the programmer intends for x. Conversely, inferring τ_1 *only* from uses of x in e_2 ignores runtime conversion obligations tied to the FFI shape τ_1'. There is no subtyping rule making arbitrary τ_1' a subtype of native τ_1 beyond the $\sim_{\mathsf{ext}}$ relation.

The standard way to recover a principal, predictable algorithm is therefore:

- treat τ_1 on the FFI `let` as *required* (or generated from a surface cast_{τ_1} with an explicit target type), and
- *check* e_1 against τ_1' such that $\tau_1' \sim_{\mathsf{ext}} \tau_1$,

while continuing to *infer* inside e_2 as in ordinary ML once $x : \tau_1$ is fixed. Equivalently, one adopts *bidirectional* typing at the boundary: the annotation τ_1 is an input (check mode) for the cast target, and an output (infer mode) for occurrences of x in e_2.

5.3 References and the native-content constraint

Rule [Ref] requires τ to be native. During inference, unification must therefore *reject* any solution that instantiates the content type of a reference with a type that mentions extern. This is not a mere equation between types; it is a *predicate* on τ ("contains no extern") that must be enforced after solving constraints, or carried as a guarded unification constraint.

5.4 `let`-polymorphism and generalization

If one extends the language with ML-style let-polymorphism (generalizing type variables at `let`), a separate design choice arises for values whose type is extern or that involve FFI boundaries. Many implementations treat such values like other *non-generalizable* artifacts (compare references in Standard ML): they are not assigned polymorphic types at `let` because their meaning may be tied to a particular call or allocation. This note does not fix a generalization policy; it only flags that extern-typed e_1 is a natural candidate for restriction if the implementation wishes to avoid silently reusing foreign handles at multiple types.

5.5 Principal types

In the core fragment without the FFI `let`, Hindley–Milner-style inference often yields *principal* types: a most general typing from which all others follow by substitution. At [Let-extern], several native choices of τ_1 can be consistent with the same τ_1' unless an annotation fixes τ_1. Thus there may be no single "best" τ_1 inferrable from e_1 alone without violating the programmer's intended cast. Requiring τ_1 at the boundary restores principal behavior *relative* to that annotation: inference proceeds in the rest of the program under a fixed contract.

5.6 Summary

Type inference for the native bulk of the language can follow standard algorithms. `extern` mainly affects the *algorithmic* story at boundaries: the cast-capable `let` needs an explicit native target type τ_1 (or an equivalent surface cast), `ref` must exclude non-native content types, and generalization at `let` may treat `extern`-bearing bindings conservatively. These choices mirror the declarative intent of the rules: foreign data is admitted only where the static semantics names a native type for the bound name, and runtime checks (when needed) are tied to that name.

6 Cangjie extern Type Specification

This section proposes a complete, plain-language specification shape for adding a core `extern` type to a Cangjie-like language. It is aligned with the official Cangjie documentation portal (`https://cangjie-lang.cn/en/docs`) and with the published Cangjie material on generics and multi-paradigm design (`https://docs.cangjie-lang.cn/docs/0.53.18/Spec/source_zh_cn/Chapter_09_Generics(zh).html`, `https://docs.cangjie-lang.cn/docs/0.53.13/white_paper/source_zh_cn/cj-wp-multiparadigm.html`). The intent is to extend that baseline with `extern` while preserving static predictability, clear diagnostics, and practical interop.

6.1 Baseline summary from Cangjie specification

The current Cangjie baseline relevant to this extension is:

- Generics are available on functions and major type declarations, with `where` constraints.
- User-defined generic types are invariant.
- Function overloading and generic argument inference are part of call resolution.
- Interface-based and class-based subtype constraints are central to generic checking.
- The language and tooling are specified through an official spec channel and conformance-oriented documentation.

6.2 Extension objective

`extern` is a boundary type for values crossing language or runtime boundaries. It is not a replacement for regular static typing. Native code may consume `extern`-compatible values only when a target native type is explicit at that boundary (annotation, explicit type argument, or explicit conversion call). Boundary failures are reported at the declared conversion point.

6.3 Mapping to baseline: compatible, extension, conflict

Compatible with baseline.

- Generic declarations and `where` syntax remain unchanged.
- Invariance of user-defined generic types remains unchanged.
- Existing overload mechanism remains; this section adds one ranking rule for conversion-requiring candidates.

Extension required.

- A new core type constructor: `extern`.
- Legality rules for placing `extern` inside generic arguments.
- Boundary conversion API requirements and runtime-failure categories.
- Additional conformance tests for compile-pass, compile-fail, and runtime-fail extern cases.

Potential conflict points to resolve.

- Whether `extern` directly satisfies nominal interface bounds.
- How overload ranking treats candidates that require boundary conversion.
- Whether mutable storage restrictions apply only directly or transitively through nested containers.

6.4 Normative candidate rules (plain language)

Rule group A: generic placement.

- `extern` may appear as a generic argument only in declarations marked as boundary-capable by the standard library or by an implementation-defined interop annotation.
- In non-boundary generic APIs, `extern` as a type argument is rejected at compile time.
- Nested appearances (for example, `Array<Box<extern>>`) follow the same rule recursively.

Rule group B: constrained generics.

- `extern` does not satisfy nominal interface bounds by default.
- To use constrained APIs, callers convert boundary values to a native constrained type before the constrained call.
- Implementations may provide explicit adapter traits, but this is opt-in and never implicit.

Rule group C: inference and overloading.

- If a call involving `extern` leaves multiple valid native targets, the call is ambiguous unless the program provides explicit type arguments or explicit target annotation.
- During overload resolution, candidates requiring boundary conversion are ranked after candidates that type-check without conversion.

Rule group D: mutable state safety.

- Mutable storage of types containing `extern` is disallowed transitively in ordinary modules.
- Boundary modules may opt in with explicit annotation, and then must perform explicit conversion at read or write points.

Rule group E: diagnostics contract.

- Static errors: illegal placement, unsatisfied constraints, ambiguous inference.
- Runtime errors: explicit boundary conversion failure.
- Implementations may vary message text, but category and source location of boundary failures must be stable.

6.5 Cangjie examples for conformance

Compile-pass: explicit conversion before constrained generic call.

```
interface Comparable<T> {
    func compare(other: T): Int64
}

func pickMax<T>(a: T, b: T): T where T <: Comparable<T> {
    return if (a.compare(b) >= 0) a else b
}

func fromBoundary(x: extern): Int64 {
    let n: Int64 = Interop.toInt64(x)
    return pickMax<Int64>(n, 10)
}
```

Compile-fail: extern used in non-boundary generic position.

```
class Box<T>(let value: T)

func bad(raw: extern) {
    let b: Box<extern> = Box<extern>(raw) // compile error in ordinary module
}
```

Compile-fail: constrained generic without prior conversion.

```
interface Hashable<T> {
    func hashCode(): Int64
}

func hashOne<T>(x: T): Int64 where T <: Hashable<T> {
    return x.hashCode()
}

func bad2(raw: extern): Int64 {
    return hashOne(raw) // compile error: extern does not satisfy bound
}
```

Runtime-fail: explicit boundary conversion failure.

```
func parseInt(raw: extern): Int64 {
    return Interop.toInt64(raw) // runtime error if raw is not integer-compatible
}
```

6.6 Open decisions and defaults

The ML-like baseline does not uniquely fix the following Cangjie-level details. This section chooses defaults and records alternatives.

- **Nominal constraints default:** `extern` does not satisfy nominal bounds directly. *Alternative:* marker-interface opt-in.
- **Overload ranking default:** no-conversion candidates win over conversion-requiring candidates. *Alternative:* equal ranking with ambiguity errors.
- **Storage safety default:** transitive restriction for mutable storage. *Alternative:* direct-only restriction.
- **Instantiation default:** keep ordinary generic specialization behavior; do not generate specialization variants only to model conversion outcomes. *Alternative:* partial sharing for extern-bearing instantiations.
- **Diagnostics default:** error categories and source locations are normative; exact text is implementation-defined.

6.7 Completeness audit checklist

This section is complete with respect to:

- **Spec surface:** grammar placement policy, generic constraints, overload/inference effects, and mutable-state policy.
- **Behavior:** compile-pass, compile-fail, and runtime-fail categories.
- **Interop:** explicit relation between core `extern` and boundary conversion APIs.
- **Risk model:** soundness, performance/code-size, and ergonomics tradeoffs are addressed by defaults and alternatives.
- **Conformance:** concrete Cangjie examples are included for each required case.

6.8 Recommended placement in a full language spec

For a production Cangjie specification, this material should appear as:

- a normative chapter on `extern` type placement and generic interaction,
- a normative chapter on boundary conversion APIs and failure behavior,
- a conformance appendix containing compile-pass, compile-fail, and runtime-fail extern tests.